



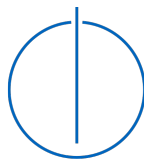
DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics: Games Engineering

Hardware-accelerated Raytracing in Dynamic Environments using Vulkan

Christian Mulser





DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics: Games Engineering

Hardware-accelerated Raytracing in Dynamic Environments using Vulkan

Hardwarebeschleunigtes Raytracing in dynamischen Umgebungen mittels Vulkan

Author:	Christian Mulser
Supervisor:	Prof. Dr. Rüdiger Westermann
Advisor:	Prof. Dr. Rüdiger Westermann
Submission Date:	September 29, 2025



I confirm that this bachelor's thesis in informatics: games engineering is my own work and I have documented all sources and material used.

Munich, September 29, 2025

Christian Mulser

Acknowledgments

I thank my supervisor, Rüdiger Westermann, for guiding me along the way and providing useful feedback to my work. I also thank Sebastian Wohner for providing access to the required modern RTX-GPUs and for quickly helping me out with technical difficulties.

Abstract

Contents

Acknowledgments	iii
Abstract	iv
1. Introduction	1
2. Related Work	3
2.1. Real-Time Ray Tracing on GPUs	3
2.2. Dynamic Scene Ray Tracing	3
2.3. Acceleration Structures and Memory Management	3
2.4. Summary and Research Gap	3
3. Technologies	4
3.1. System Specifications	4
3.2. Build Tools	4
3.3. Graphics API	4
3.4. Programming Language	4
3.5. File Formats	5
3.5.1. glTF	5
3.5.2. CSV	5
3.6. Libraries	5
3.6.1. Volk	5
3.6.2. VMA (Vulkan Memory Allocator)	5
3.6.3. vk-bootstrap	6
3.6.4. cgltf	6
3.6.5. CLI11	6
4. Implementation	7
4.1. Vulkan Abstraction	7
4.1.1. Vulkan Handles and Move Semantics	7
4.1.2. Shared Pointers	7
4.1.3. The Builder Pattern	8
4.1.4. Basic Vulkan Object Structure	8
4.1.5. The Vulkan Context	10
4.2. Raytracing-related Objects	12
4.2.1. Acceleration Structures	12
4.2.2. Raytracing Pipeline	13

4.2.3. Shader Binding Table	15
4.3. Scene	17
4.3.1. glTF Structure	17
4.3.2. Scene Structure	18
4.3.3. Updating a Scene	22
5. Results and Discussion	25
6. Conclusion	26
A. General Addenda	27
A.1. Detailed Addition	27
B. Figures	28
B.1. Example 1	28
B.2. Example 2	28
List of Figures	29
List of Tables	30
Bibliography	31

1. Introduction

For a long time, **rasterization** has been the standard for rendering 3D scenes in real-time (e.g. in video games). The basic steps of rasterization are as follows:

1. Every vertex of a 3D object gets transformed in the vertex shader.
2. The transformed triangles are projected onto a 2D pixel grid. For every pixel that the triangle covers, a fragment is generated.
3. The fragment shader runs for every generated fragment. It uses the data of the fragment as well as some global data (e.g. light sources) to determine the final color of the pixel.

Rasterization is very fast and efficient, but it does have its downsides as well. When determining the color of a pixel, we only have access to the data in the current fragment and some global data. The fragment is not aware of other objects in the scene. This makes effects like shadows and reflections very difficult and they require a workaround like shadow-mapping and screen-space-reflections. In recent years, a new way to do real-time rendering slowly emerged, which of course is **ray tracing**.

Ray tracing works by casting a ray through the pixels of your screen into the 3D scene, to determine the surface to be displayed. When you hit a surface, you can cast secondary rays such as shadow rays, reflection rays or refraction rays. This is much closer to real life, where light rays bounce between objects until they reach our eyes or a camera lens and because of that it is much easier to generate realistic looking images. However, it was never suitable for real-time applications in the past due to enormous amount of processing necessary.

Even though ray tracing is relatively new in the realm of real-time graphics, the concept of ray tracing is almost as old as rasterization. In 1968 Arthur Appel explores the idea of casting a ray through a pixel to determine, which surface is visible, and also the idea of casting a ray from a surface point to the light source to check, if the light reaches the surfaces unobstructed [App68]. Later in 1980 Turner Whitted introduces the well-known ray tracing model known as "Whitted-Raytracing" [Whi80]. If a surface is hit by a ray, additional rays are cast for reflection, refraction and shadow. In the 1990s ray tracing started to be used in visual effects for movies like "Terminator 2: Judgment Day" (1991) or "Jurassic Park" (1993). In the 2010s animation studios like Pixar move to fully path traced rendering [Chr+18], which uses ray tracing to trace the whole path from a light source to the camera in order to generate photorealistic images. An example for this is Pixar's animated movie "Finding Dory" (2016) [HVF16].

In 2018 Nvidia paved the way for ray tracing in real-time applications by introducing the RTX-Series. These graphics cards used the "Turing architecture", which introduced hardware-accelerated ray tracing with the new RT Cores. RT Cores traverse the BVH (Bounding Volume

Hierarchy) autonomously, and by accelerating traversal and ray/triangle intersection tests, they offload the SM (Streaming Multiprocessor), allowing it to handle other vertex, pixel, and compute shading work [Cor18].

Having to test every ray against every triangle for intersection is not feasible at all and takes way to much processing power, even for non-real-time applications. A much smarter way to find the triangles, that are hit by a ray, is using a bounding volume hierarchy (BVH). It is a tree structure of volumes (usually AABBs), which devide the scene into smaller and smaller subsections as we go down the tree. We can prune a branch as soon as we realize, that the section does not intersect with the ray and therefore avoid a lot of ray/triangle intersections.

In this thesis we will explore the topic of ray tracing in dynamic environments. We are going to look at objects, that move through a scene, but also at objects that deform their mesh using skeletal animation. Specifically, we are going to use the Vulkan API and look at how it implements ray tracing support. We are going to learn how to create the raytracing pipeline, setup our shader binding table and build our acceleration structures as well as the different ways we can update them later during an animation. We are then going to test how different scenes perform using different approaches and compare them to each other.

2. Related Work

2.1. Real-Time Ray Tracing on GPUs

2.2. Dynamic Scene Ray Tracing

2.3. Acceleration Structures and Memory Management

2.4. Summary and Research Gap

3. Technologies

3.1. System Specifications

Operating System	Ubuntu 24.04.2 LTS
CPU	Intel(R) Xeon(R) W-2235 CPU @ 3.80GHz
GPU	NVIDIA GeForce RTX 3090

Table 3.1.: System Specifications

3.2. Build Tools

- Premake 5
- GNU Make 4.3

The project uses Premake 5, which does not build the project directly, but it is a command line utility which reads a scripted definition of a software project and uses it to generate project files for toolsets like Visual Studio, Xcode, or GNU Make. It is a simpler alternative to CMake.

We use Premake to create Makefiles, which then ultimately compile our project.

3.3. Graphics API

As the graphics API for our project we chose **Vulkan 1.2**. Vulkan is a modern platform-agnostic API to handle GPU operations. It is the successor to OpenGL, which was discontinued in 2016. Vulkan allows for much more fine-grained control over the GPU, supports multi-threading and for this project most importantly raytracing, through extensions like `VK_KHR_ray_tracing_pipeline` and `VK_KHR_acceleration_structure`. Since these extensions were introduced in Vulkan 1.2, that is the version used in the project.

3.4. Programming Language

At the start of the project Rust was considered as the projects programming language. Rust is a new, memory-safe, low-level programming language and a modern alternative to languages like C and C++. While Rust does have a lot of advantages over C++, we chose **C++** in the

end. The main reason for this is that the Vulkan API was designed for C/C++. There are a lot of great Vulkan-libraries for Rust, like the popular "Ash" create, which is generated from "vk.xml". However, way you use the API does differ from the classic C API. This could be problematic, considering that nearly all the few resources relating to raytracing in Vulkan use C/C++. This could have made it difficult to translate the raytracing code into Rust later down the line, so C++ was the safer choice.

3.5. File Formats

3.5.1. glTF

The dynamic 3D scenes, that the project uses, are stored in the **glTF-2.0** file format. It stores its models in a scene graph and supports skeletal animation as well as morph animations. By default, it stores the scene data in the popular JSON format. Additionally, buffers are stored in binary in BIN files and textures are stored in image files (PNG, JPEG, ...). Since the file format was developed by Khronos Group, which is also behind Vulkan, the data inside the buffers is laid out in a way that makes it easy to use in Vulkan.

3.5.2. CSV

CSV stands for Comma Separated Values and is used to store tabular data. It is a text-based format and therefore very easy to use. Each line of text inside a CSV file represents a row and the values of that row are separated by commas. We use it to store our benchmarks in our projects. We can later import a CSV file in an application like Excel to create graphs and analyse our gathered data.

3.6. Libraries

3.6.1. Volk

volk is a meta-loader for Vulkan. It allows you to dynamically load entry points required to use Vulkan without linking to vulkan-1.dll or statically linking Vulkan loader. Additionally, volk simplifies the use of Vulkan extensions by automatically loading all associated entry points [<https://github.com/zeux/volk>].

3.6.2. VMA (Vulkan Memory Allocator)

Memory Management in Vulkan is quite difficult. For example, when you create a buffer or image in Vulkan there is no memory associated with it. The programmer needs to manually allocate block of memory with a certain type from a certain memory heap. The properties of these memory heaps depend on the underlying physical device and need to be queried first. Additionally, creating a new memory allocation for each resource may be suboptimal, since the number of allocations can be very limited and many allocations may negatively impact

performance. VMA (Vulkan Memory Allocator) simplifies, automates and optimizes memory management in Vulkan.

3.6.3. vk-bootstrap

Initializing Vulkan involves a lot of boilerplate code, which is going to be basically the same in every Vulkan application. VkBootstrap can be used to reduce the amount of boilerplate code in a project.

This library simplifies the tedious process of:

- Instance creation
- Physical Device selection
- Device creation
- Getting queues
- Swapchain creation

[Lun25]

3.6.4. cglTF

To load a glTF file in the project, cglTF is used. cglTF parses the glTF file, which is stored in JSON. It would also have been possible to read the JSON directly, but cglTF takes care of validating the file, loads the required buffers from their BIN-files into memory and creates a C-structure, which is a lot more comfortable to use. Our project does use not this structure directly, but rather used to generate a Scene object.

3.6.5. CLI11

CLI11 is a header-only library written in modern C++, that is designed to parse the command-line options of your application. It is simple, feature-rich and safe. CLI11 allows you to define flags, options, commands and subcommands. You can also add certain restrictions to your parameters, like only allowing positive numbers or that a parameter is an existing path.

4. Implementation

4.1. Vulkan Abstraction

4.1.1. Vulkan Handles and Move Semantics

As already mentioned in the Methods and Technology chapter, Rust was originally considered as programming language, because it is modern and memory safe. One of Rust's core features is ownership, which means that each value/object can only be owned and managed by a single variable, and when that variable goes out of scope, it alone is responsible for cleaning up the object. When that variable gets assigned to another variable, the new variable becomes the new owner (i.e. the value gets moved). Ownership would be a very nice feature to manage Vulkan objects. Vulkan objects should not be copied on assignment but only moved and when the object is no longer needed (i.e. the owner goes out of scope), it should automatically be destroyed.

While C++ does not support ownership directly, it does have so-called move semantics, with which we can emulate ownership. The ownership of Vulkan handles is managed by the `Handle<T>` class. To do this the class declares a default constructor, move constructor and destructor and overrides the move assignment operator. It is also important, that the copy constructor and copy assignment operator are deleted. The reason for that is that copying a Vulkan object is usually very expensive or not even possible and it is almost never necessary. When we want to copy an object, we need to explicitly create a new one first. This avoids unnecessary and complicated accidental creation and copying of new objects.

4.1.2. Shared Pointers

Since C++11 the language supports so-called smart pointers, which make memory management safer and less complicated. One of these smart pointers is `std::shared_ptr<T>` (or our alias `ptr::Shared<T>`). This type of pointer allocates an object, which can then be shared between multiple owners. The object lives while it has at least one owner, and the last object to go out of scope cleans up the object. This structure can be combined very nicely with our previously defined handle. A Vulkan object can only have one owner, which is a harsh limitation and often causes problems. If we wrap the object in a shared pointer, it can be shared across multiple owners. We also added the class `ToShared<T>`, which a class can inherit from. An instance of that class can then be moved into a shared pointer by calling `to_shared()`.

4.1.3. The Builder Pattern

As mentioned, before we want the creation of Vulkan objects to be very explicit. The first method to create objects in C++ you think of are constructors, but they prove to be problematic for a couple of reasons. The first problem is that constructors are often called implicitly (e.g. the default constructor gets called when declaring a variable). Default construction only makes sense for very few Vulkan objects (like for example a `VkFence`) and we do not want a Vulkan object to be created when implicitly (or explicitly) calling the default constructor. So, for that reason, the default constructor simply does not create a Vulkan object or, in other words, leaves the handle at `VK_NULL_HANDLE`. Now if a different constructor actually creates an object, the behavior of constructors becomes inconsistent regarding Vulkan object creation. In addition to that, Vulkan objects are almost always created using a `*CreateInfo` struct, which may take a lot of parameters. To solve all of these issues we implemented a `Builder` class for most Vulkan objects. Some exceptions are `Fence` and `ImageView`, which are created with `Fence::create` and `ImageView::create_from` respectively. This makes it very clear when a Vulkan object is created, since it can only be created using a `Builder` or a static method (like `Fence::create`) and especially it can never be created by a constructor. This is also the reason Vulkan objects never have constructor, besides the default constructor.

```
vk::Buffer buffer; // declare a buffer (handle is VK_NULL_HANDLE)

buffer = vk::BufferBuilder()
    .add_usage(VK_BUFFER_USAGE_TRANSFER_DST_BIT)
    .add_usage(VK_BUFFER_USAGE_VERTEX_BUFFER_BIT)
    .memory_usage(VMA_MEMORY_USAGE_GPU_ONLY)
    .size(128)
    .build();
```

4.1.4. Basic Vulkan Object Structure

All of the Vulkan objects follow a basic structure, which can be seen in the following code sample:

```
class Foo : public utils::Move, public ptr::ToShared<Foo> {
    friend class FooBuilder;
private:
    Handle<VkFoo> foo_handle{};
    // Additional member variables

public:
    Foo() = default;

    inline VkFoo handle() const { return foo_handle; }
    // Additional getter functions

    // Additional functions
};

class FooBuilder {
public:
    using Ref = FooBuilder&;
private:
    // build parameters, e.g. VkFormat _format;

public:
    FooBuilder() = default;

    // parameter setter functions, e.g. Ref format(VkFormat format);

    Foo build() const;
}
```

Figure 4.1.: An example of how to use command buffers in your code.

It is important to note, that `Foo` is just a placeholder for an actual Vulkan object (like `Buffer`, `Image` or `Pipeline`). Usually, the name of the class is just the name of the Vulkan handle without the "Vk" prefix (e.g. `VkBuffer` $\implies$ `Buffer`), but there are some exceptions. For example a `Shader` class holds a `VkShaderModule` handle. As we can see, the Vulkan handle is wrapped in the previously mentioned `Handle<T>` class, which ensures correct move semantics. The member function `handle` allows us to access the raw Vulkan handle.

The class inherits from `utils::Move` and `ptr::ToShared<Foo>`. The `utils::Move` class implements the default move constructor and move assignment, and it deletes the copy constructor and copy assignment. The `ptr::ToShared<Foo>` class contains the function `to_shared`, which is just syntax sugar for moving the object into a shared pointer.

The additional variables in the class are often used to store meta-data (e.g the format and extent of an image) or to keep other objects alive in a shared pointer, that are referenced by the underlying object (e.g. the render pass of a pipeline). There usually is also an getter function for these variables.

The additional functions include functions, that actually use or manipulate the underlying object. Often, these functions have a "cmd_" prefix, which means that they are wrappers around one or multiple commands (e.g. `Buffer::cmd_copy_into`).

Even though not every Vulkan object has uses a builder class, it is also shown in the sample code, since most Vulkan objects use it.

4.1.5. The Vulkan Context

Before we can actually do any work in Vulkan, we need to do a lot of setup. This usually includes the following steps:

1. Create a `VkInstance`.
2. Select a `VkPhysicalDevice`, which supports the extensions, features and capabilities required by the project.
3. Create a `VkDevice` from the selected `VkPhysicalDevice`.
4. When creating the `VkDevice`, `VkQueue`s from a certain queue family need to be enabled, to be able to submit command buffers later.

These Vulkan objects are accessed very frequently throughout the project. For example, you need to pass the `VkDevice` as a parameter when creating or destroying most Vulkan objects and a command buffer, which executes a sequence of Vulkan commands, needs to be submitted to a certain `VkQueue`. Keeping track of all of these Vulkan objects is quite tedious, which is why we introduced the Vulkan context. It is represented by the `Context` class, which stores these Vulkan objects, allows the programmer to access them anywhere and reduces the effort of setting up Vulkan.

The Vulkan context contains the following objects:

- the `VkInstance`
- the `VkPhysicalDevice` and `VkDevice`
- an optional `GLFWwindow*` from which a `VkSurfaceKHR` is created and also stored (this is necessary for rendering to a window, but it will not be used in our main application, since it is headless)
- the `CommandManager` (manages queues and command pools)
- the `VmaAllocator` (used when allocation memory)

The class is designed similar to a singleton, but it deviates in some parts. A singleton usually creates its instance at the start of the application or when it is used for the first time (lazy instantiation). However, `Context` can not be implicitly instantiated this way, because it needs some parameters stored in a `ContextInfo`, when being instantiated. For this reason we need to explicitly create the Vulkan context using `Context::create(...)` with the desired `ContextInfo` before calling `Context::instance()` for the first time or we will get a `nullptr`. While all other objects in our project get cleaned up automatically when going out of scope, the Vulkan context also needs to be destroyed explicitly using `Context::destroy()` when it is no longer needed. If we now want to get the current `VkDevice`, we can call `Context::instance()->get_device()`.

4.2. Raytracing-related Objects

In this section we will take a closer look at all the objects related to ray tracing and our abstractions of them.

4.2.1. Acceleration Structures

We are going to start with the acceleration structures. Acceleration structures are data structures used by the implementation to efficiently manage scene geometry as it is traversed during a ray tracing query [Khr25]. The acceleration structure has two levels:

- **Bottom Level Acceleration Structure (BLAS)**

The lower level, the bottom-level acceleration structure, contains the triangles or axis-aligned bounding boxes (AABBs) of custom geometry that make up the scene [Koc+20]. This usually corresponds to a mesh, that is used in a scene.

- **Top Level Acceleration Structure (TLAS)**

The upper level, the top-level acceleration structure, contains references to a set of bottom-level acceleration structures, each reference including shading and transform information for that reference [Koc+20]. This represents a scene filled with objects, that each can have a different transformation and the mesh.

The relationship between TLAS and BLAS can be seen in this figure:

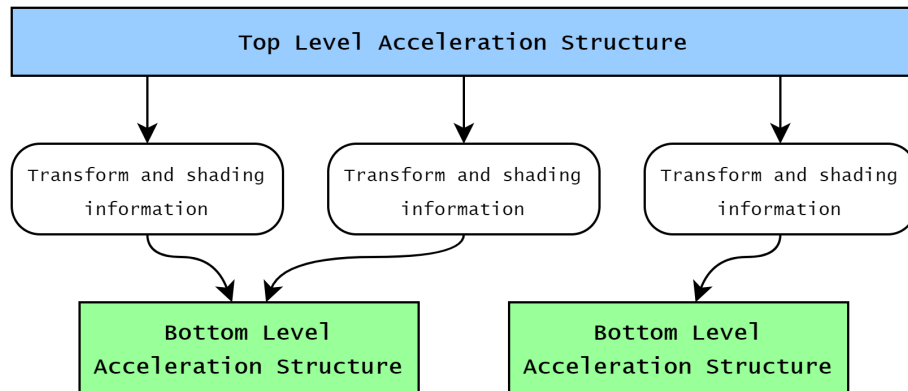


Figure 4.2.: Acceleration structure

Vulkan represents both types of with the `VkAccelerationStructureKHR` handle. This is however just a handle with some meta-data. We need a `VkBuffer` to store the actual data of the acceleration structure (e.g. the BVH). Before building an acceleration structure we need to reference the data used for building in a `VkAccelerationStructureGeometryKHR`. For the BLAS this is usually a list of triangles (even though other primitives are also possible)

and for the TLAS it is a list of instances, with transform and shading information as well as a handle to a BLAS. Once our `VkAccelerationStructureGeometryKHR` is ready we can query for the `VkAccelerationStructureBuildSizesInfoKHR`, which contains the variable `accelerationStructureSize`, which is the size for the buffer, that stores the acceleration structure, as well as the variables `updateScratchSize` and `buildScratchSize`, which are the required sizes of a temporary scratch buffer which is only used during building/updating the acceleration structure. Now we can build an acceleration structure using the command `vkCmdBuildAccelerationStructuresKHR`.

We encapsulate a BLAS and TLAS in the classes `Blas` and `Tlas` respectively. They both have the following member variables:

- `blas/tlas`: The `Handle<VkAccelerationStructureKHR>`.
- `buffer`: The buffer that stores the data for the acceleration structure.
- `build_scratch_buffer`: The scratch buffer used during building.
- `update_scratch_buffer`: The scratch buffer used during updating/refitting.

These classes also use a builder (see subsection 4.1.3 and subsection 4.1.4). The builder expects a list of `BlasGeometry`s when building a `Blas` and a list of `TlasInstances` when building a `Tlas`. A `BlasGeometry` describes the vertex buffer and optional index buffer used for a triangle mesh. A `TlasInstance` on the other hand stores a reference to a BLAS in a `ptr::Shared<Blas>` and a transformation. The builder also has the property `dynamic`, which needs to be set to `true`, if you want to refit the acceleration structure later. It also has the property `fast_build`, with which you can prefer a fast build over a fast trace.

When we want to update an acceleration structure, there are two functions we can use:

- `rebuild`:
This function recreates the BVH from scratch. It essentially does the same thing as the `build` function in the builder, except that it uses the already created handle and buffers and does not create them again.
- `refit`:
This function does not the structure of the BVH. Instead it only adjusts the bounds of the AABBs in the BVH for the new vertex positions. This is faster than rebuilding from scratch, but can lead to a sub-optimal BVH, which results in higher ray tracing durations.

4.2.2. Raytracing Pipeline

In order to issue ray tracing commands we need a ray tracing pipeline. The following figure shows a flow chart of the ray tracing pipeline.

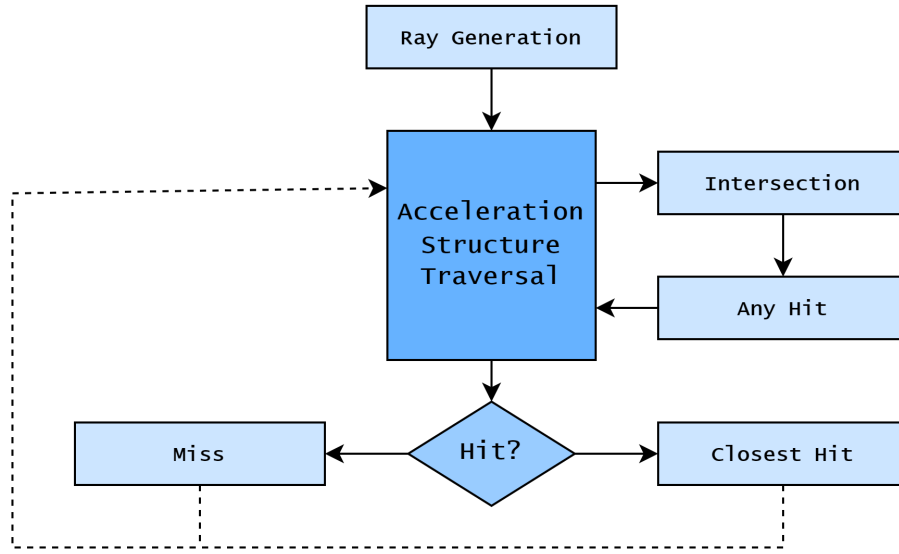


Figure 4.3.: Ray tracing pipeline

A raytracing pipeline in Vulkan stores a list of shader groups. A shader group consists of one or more shaders and in our application we are interested in two types:

- **General Shader Group**

A general shader group only stores a single "general" shader. It can call its shader independently of any ray/triangle intersections or hits. We use this type of shader group in the ray generation and miss stage of the pipeline.

- **Triangle Hit Shader Group**

A triangle hit shader group contains the any-hit shader and the closest-hit shader, which can be called when a ray/triangle intersection occurs. It calls the any-hit shader for every ray/triangle intersection along the ray and the closest-hit shader for the first intersections, if the ray intersects a triangle at least once.

The class `ShaderGroup` implements shader groups in our application. Each `ShaderGroup` contains a unique identifier, which can be accessed using the function `id`. It contains either a `ShaderGroupGeneral` or a `ShaderGroupHit` struct inside a `std::variant`. A `ShaderGroupGeneral` contains the shader `general`, while a `ShaderGroupHit` contains the shaders `closest_hit` and `any_hit`.

The raytracing pipeline is contained in the class `RtxPipeline`. The `RtxPipeline` is created from a list of `ShaderGroups`. The `ShaderGroup` objects are just descriptions of the shader groups. The actual shader groups get allocated by the raytracing pipeline and can be accessed through an array of shader group handles. The `RtxPipeline` stores a map from the `id` of every `ShaderGroup` to the index of the shader group handles for easier access to the handles.

The raytracing pipeline also needs a pipeline layout, which is stored in a `PipelineLayout` object and it needs to set the maximum recursion depth of rays in `max_ray_recursion_depth`.

4.2.3. Shader Binding Table

Once the ray tracing pipeline is ready we need to create a shader binding table (SBT), which selects the correct shader group in every stage of the raytracing pipeline.

Ray Generation	Ray Gen Group		
Miss	Miss Group	Miss Group	
Hit	Hit Group	Hit Group	Hit Group
Callable	Callable Group		

Figure 4.4.: Shader Binding Table

As we can see it has 4 rows:

- **Ray Generation:** Shader group used for ray generation.
- **Miss:** Shader groups that can be used when a ray misses.
- **Hit:** Shader groups that can be used when a ray intersects with at least one object.
- **Callable:** Shader groups, which contain arbitrary shaders that can be called by other shaders and are not tied to a specific ray traversal event. (Not used in our application)

The shader binding table needs to be stored in a `VkBuffer` and the rows of the table are represented by a `VkStridedDeviceAddressRegionKHR`, which references a region inside that buffer, and specifies a certain stride, which specifies how many bytes we need to get from one element to the next in that region.

When casting a ray in a shader using the function `traceRayEXT`, we can specify a miss offset, which is the offset of the shader group in the miss row, that should be used when the ray misses. In a similar way we can also define an offset into the hit shader groups our ray should use when a hit occurs. Every instance in the TLAS can also define an additional offset into the hit shader group, which gets added to the previously mentioned offset.

Because the SBT is strongly dependent on the raytracing pipeline, it does not use a builder, but rather we build it with the member function `build_sbt` inside `RtxPipeline` and a `SBTInfo` struct as a parameter. `SBTInfo` has the following member variables:

- `ray_gen_group`: The `ShaderGroup` used for ray generation.
- `miss_groups`: A list of `ShaderGroup`s, which can be used when a ray misses.
- `hit_groups`: A list of `ShaderGroup`s, which can be used when a ray hits.

The `ShaderGroup`s used inside the `SBTInfo` must be a sub-set of the `ShaderGroup`s used to create the ray tracing pipeline. When building the SBT we first retrieve the array of shader group handles of the ray tracing pipeline using `vkGetRayTracingShaderGroupHandlesKHR` and with the mapping from shader group ID to index we can get the index of the handle for every shader group. We then store the ray generation shader group handle, followed by the sequence of miss shader group handles and finally the sequence of hit shader group handles in a buffer. Finally, we set the `VkStridedDeviceAddressRegionKHR` for every row and our SBT is completed. We can now use our table rows when calling `vkCmdTraceRaysKHR`.

In the following code sample we can see how to create the ray tracing pipeline and SBT:

```
ShaderGroup ray_gen_group = ShaderGroup::create_general(ray_gen_shader);
ShaderGroup miss_group = ShaderGroup::create_general(miss_shader);
...
ShaderGroup hit_group = ShaderGroup::create_hit_closest(closest_hit_shader);
...

RtxPipeline pipeline = RtxPipelineBuilder()
    .add_shader_group(ray_gen_group)
    .add_shader_group(miss_group)
    .add_shader_group(hit_group)
    ...
    .layout(pipeline_layout)
    .max_ray_recursion_depth(1)
    .build();

SBT sbt = pipeline.build_sbt(SBTInfo()
    .ray_gen_group(ray_gen_group)
    .miss_groups({ miss_group, ... })
    .hit_groups({ hit_group, ... }));
```

4.3. Scene

In order to understand, why we structured our scenes the way we did, we first need to take a look at the glTF file format, that stores our scenes. As already mentioned (see subsection 3.5.1).

4.3.1. glTF Structure

A glTF stores its data in the JSON format. There different types of objects if glTF, each of them is stored in an array. An objects often references an other object by its index in the respective array. The objects we are interested in are:

- **Buffer, Buffer view and Accessor**

These three objects work together closely. A buffer stores a block of binary, usually contained in a separate BIN file. A buffer view describes a certain part of a buffer. It references a certain buffer and contains an offset, length and optional stride. A buffer view sees the data as just raw data. To associate some kind of meaning to the data we use an accessor. It references a buffer view with a certain offset and also contains the number of elements, as well as the type (e.g. scalar, vec3, mat4, ...) and component type (e.g. float, unsigned int, ...). Accessors are later used for things like vertex attributes and joint matrices.

- **Mesh**

A mesh stores a static 3D model. It contains a material and an array of primitives. Each primitive has a mode (e.g. points, lines, triangles), and optional accessor for indices and a set of accessors used for the different vertex attributes (e.g. positions, normals, uv coordiantes, vertex colors, joint weights).

- **Skin**

A skin is used for skeletal animation of a mesh. It contains an array of nodes, that are used as the skeletons joints, and an accessor for the inverse bind matrices of the joints.

- **Node**

This represents a node in the scene graph. A node always has a transform, which is either represented by a 4x4 matrix or a scale(vec3), rotation(quaternion) and translation(vec3). A node can have an array of child nodes, which inherit the transform of the parent node. In other words, a nodes transform is applied in the parent nodes coordinate system.

A node can optionally also contain a mesh, which is gonna be transformed by the nodes transform. If a mesh is present, the can also contain a skin. The nodes skin will deform the nodes mesh during an animation.

- **Scene**

A scene contains an array of root nodes. These nodes as well as all of their children are part of that scene.

- **Animation**

An animation has an array of channels and an array of samplers. A channel describes, which property is being sampled during the animation. It contains the effected node and a path the effected property of the node (e.g. "transform", "scale", "rotation"). It also holds a reference to one of the animations samplers.

A sampler is used to calculate the value of a property at a certain point in time from a set of key frames. It contains two accessors, one for the time of the key frames and one for the value of the keyframes. Additionally, a sampler contains the type of interpolation, that should be used to sample a value.

There are also some other objects (e.g. materials, textures and cameras), but since we don't use them for our project, we don't explain them here.

4.3.2. Scene Structure

We will now take a look at the structure of our scene implementation. For this we will look at the different C++ classes we used.

The `Scene` class

The `Scene` class represents a scene that can be loaded from a glTF file. It contains an array of nodes, an array of animations and an array of skins. Additionally, it stores the TLAS, that is used for the ray traversal through the scene, and the different buffers used by the scene in a `SceneBuffers` struct.

A scene can be loaded using the static method `Scene::load`, with the path to the glTF file as parameter. A scene also contains the member function `Scene::iter`, which returns a node iterator for all of the nodes contained in the scene. There also is the function `Scene::build_acceleration_structures`, which we need to call on a scene to build all of the necessary acceleration structures so we can use ray tracing. The rest of the scenes functions are explained in the next subsection about updating a scene (see subsection 4.3.3).

The `Node` class

The `Node` class directly represents a node in glTF. As we mentioned previously the transform of a node can either be stored as a matrix or separated into a translation, rotation and scale. To model that in our application, we added the struct `RawTransform`, which contains the transform, rotation and scale. We also have the struct `NodeTransform`, which stores the local transform of a node and looks like this:

```
struct NodeTransform {  
    bool raw;  
    union {  
        glm::mat4 matrix;
```

```

    RawTransform raw;
} transform;
}

```

The union `transform` is used to store either a `glm::mat4` or a `RawTransform`. The boolean `raw` is used to determine, how the union should be interpreted, i.e. if `raw` is `true`, the union contains a raw transform, otherwise it contains a matrix. The node also stores the global transform as a 4x4 matrix, but it is only valid after calling `Node::update_global_transform`.

A node also stores an array of child nodes and it can also contain a mesh. If a mesh is present, it also stores a BLAS and the node must generate a TLAS instance that references the nodes BLAS when building the acceleration structure. After building the acceleration structure for the first time, the node also stores the index of the TLAS instance, which is necessary when updating the TLAS instance later. If the node does not have a skin, the node's BLAS is the the same as the one inside the mesh.

If the node does however contain a skin, it stores an array of sub-buffers for dynamic positions. For every primitive of the node's mesh, a sub-buffer for dynamic positions needs to be present in the array. Each sub-buffer needs to have the same size as the sub-buffer for the static positions of the respective primitive. These dynamic positions are used to store the vertex positions of a mesh after skinning. If a skin is present, the BLAS is not simply just a reference the meshes' BLAS, but it is a separate, dynamic BLAS, which can be updated.

The `Mesh` class

The `Mesh` class represents a glTF mesh. It stores an array of `Primitives`. Each primitive stores three sub-buffers: `positions` for the vertex positions, `indices` for the vertex indices and `joint_weights` for the joint weights used for skinning the mesh. The positions and indices are pretty straight forward and just store a sequence of `vec3s` and `unsigned ints` respectively. The joint weights are a bit more complex, because they are stored in a 2D array of `JointWeight` structs. A `JointWeight` contains the index of the node in the skeleton of the skin and its weight. All the arrays of joint weights for a vertex are stored in sequence like shown in this figure:

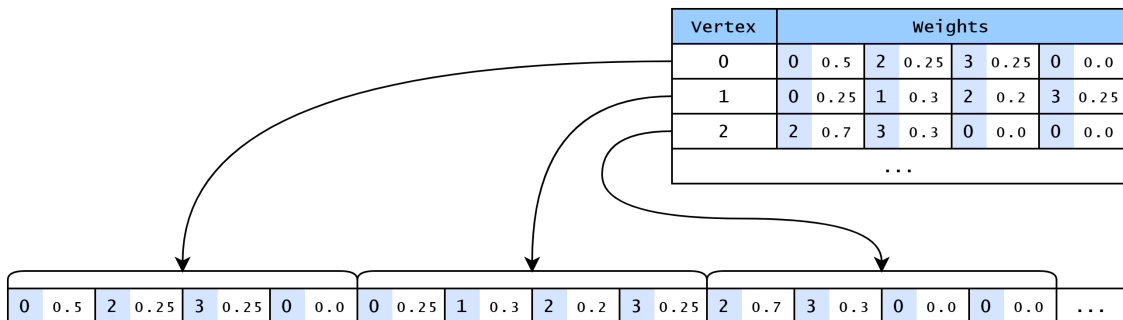


Figure 4.5.: Joint weight layout

Additionally, each primitive also contains the member variables `positions_cpu` and `joint_weights_cpu`, which store the same data as `positions` and `joint_weights`, but this time inside `std::vector`s for easier access in CPU skinning.

Finally, a mesh can also contain a BLAS, which is created for the static mesh. It is only created if a mesh is used statically by a node, i.e. a node that uses the mesh without a skin.

The `Skin` class

The `Skin` class is relatively simple and matches the glTF skin object almost exactly. It simply stores an array of `Nodes`, which represent the joints in the skeleton and an array for the inverse bind matrices, with one entry for each joint.

The `Animation` class

The `Animation` class represents a glTF animation. It contains an array of `AnimationSamplers`, which contain a reference to the node, that they modify during animation, and also the virtual function `AnimationSampler::apply_for`. This function samples a value at the specified point in time and applies it to a certain property of the nodes transform. There are three sub-classes of `AnimationSampler`, which override the `apply_for` function:

- `TranslationSampler`: samples a 3D vector and sets it as the nodes translation.
- `RotationSampler`: samples a quaternion and sets it as the nodes rotation.
- `ScaleSampler`: samples a 3D vector and sets it as the nodes scale.

The animation itself also has an `apply_for` function, which iterates over all animation channels and applies them for the given point in time.

Buffers and Sub-Buffers

glTF uses buffers, buffer views and accessors. We don't use the data in the glTF buffers directly, but we read the data and lay it out in a more convenient way. A glTF accessor may access data, that is interleaved with other data, or sparse data. Additionally, the accessor's component type may not match our component type. However, we want to have our data lined up in memory sequentially and of a fixed type. Luckily, the `cgltf` library offers the function `cgltf_accessor_unpack_floats`. This function reads all the values of the accessor, converts the components of the values to `float` and stores the values sequentially in a target buffer. The function `cgltf_accessor_unpack_indices` does the equivalent for `unsigned ints`.

We create one buffer for each usage type of our data. We have one buffer for the vertex indices, one for static vertex positions of the mesh and one for dynamic vertex positions, that may be different for each mesh. An attribute can then reference a sub-subsection into one of these buffers using a sub-buffer. A sub-buffer has a reference to the buffer of interest using a `ptr::Shared<vk::Buffer>`, as well as an offset and a length. We do not need to store the type of the elements in the sub-buffer, since the type is fixed and can be determined by

the variable. We know that the sub-buffers `positions` in `Mesh` and `dynamic_positions` in `Node` store a sequence of `vec3`s, while the sub-buffer `indices` in `Mesh` stores a sequence of unsigned ints.

The following figure gives a little bit more insight, of how these sub-buffers are used.

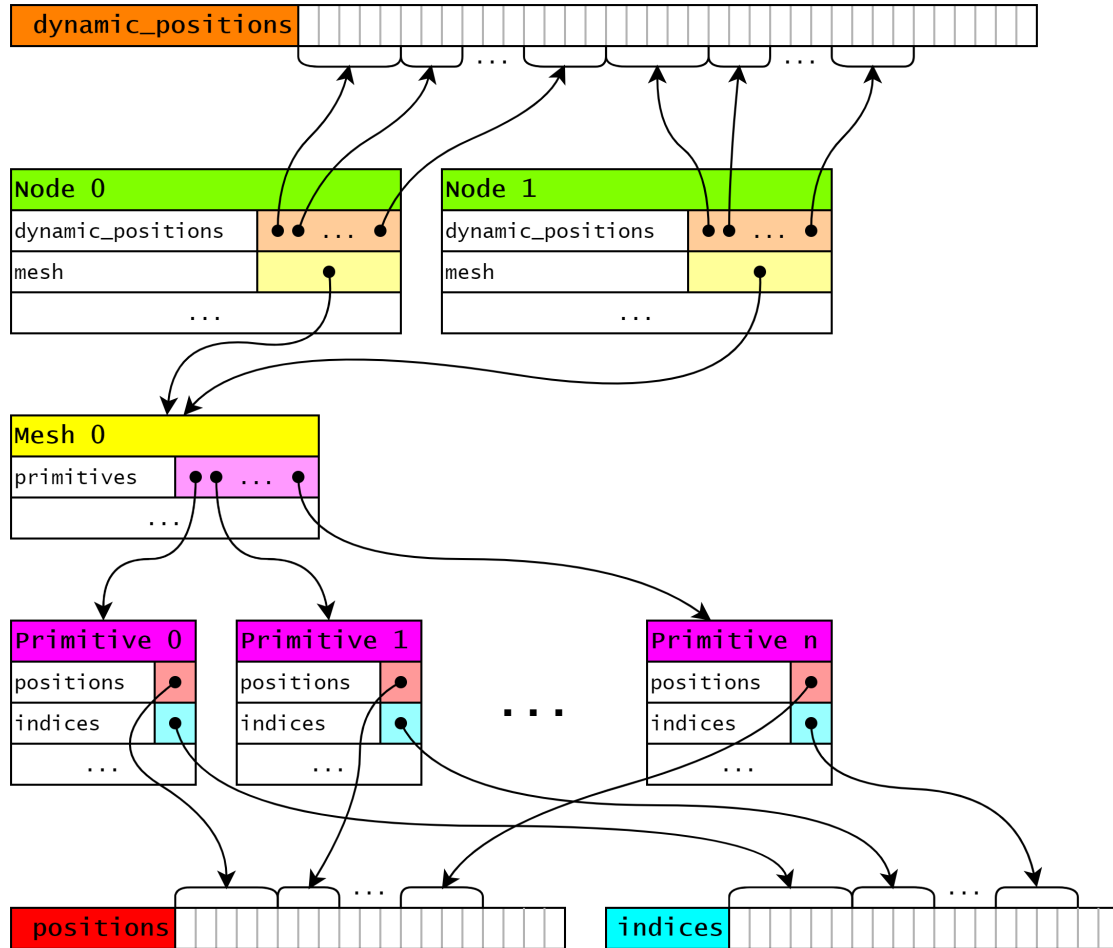


Figure 4.6.: Usage of sub-buffers in a scene

In this graphic we can see, how both nodes `Node 0` and `Node 1` reference the same mesh `Mesh 0`. This means, that they use the same sets of static positions and indices. But because two nodes with the same mesh can use different skins, each node needs to store their own set of dynamic positions. We can also see how the size of every `dynamic_positions` sub-buffer matches the size of the respective sub-buffer in `positions`.

4.3.3. Updating a Scene

Since our scenes are dynamic, we need a way to update them. We need update our scene after calling `Animation::apply_for` from an animation in our scene. A scene is updated in three steps:

1. updating the global transform for every node
2. skinning all animated models
3. updating the acceleration structure

When we want to update our scene we need to perform all of the above steps in the exact order. We keep the steps separate however, because we can use different implementations for some steps. For example, we can skin our models on the CPU or on the GPU and we can update our acceleration structure by either rebuilding it or refitting it.

Updating Global Transform

Using the member function `Node::update_global_transform` we can update the global transform of a node. This function takes in a `glm::mat4` as argument, which represents the parents transform. This argument is optional and gets set to the identity matrix by default. The following pseudo code shows the functions behaviour:

Algorithm 1 Pseudo code for updating the global transform

```
1: function UPDATE_GLOBAL_TRANSFORM(node, parent_transform)
2:   node.global_transform  $\leftarrow$  parent_transform  $\times$  node.transform
3:   for all child in node.children do
4:     update_global_transform(child, global_transform)
5:   end for
6: end function
```

The global transform is calculated by pre-multiplying the parents transform with the local transform of the node. We then call `Node::update_global_transform` recursively for every child node, with the new global transform of the current node as a parameter.

Skinning

Skinning describes the process of applying the deformation of a skeleton to a certain model. To do that we first need to calculate the joint matrices:

Algorithm 2 Calculating joint matrices

```

1: for  $i \leftarrow 0$  to joints.count - 1 do
2:   joint_matrices[i] = inverse(node.global_transform)
      × joints[i].global_transform
      × inverse_bind_matrices[i]
3: end for

```

After having calculated the joint matrices, we can skin the model:

Algorithm 3 Skinning a model

```

1: for  $j \leftarrow 0$  to (positions.count - 1) do
2:   matrix  $\leftarrow 0_{4 \times 4}$ 
3:   for all joint_weight in joint_weights[i] do
4:     matrix  $\leftarrow$  matrix + joint_weight.weight × joint_matrices[joint_weight.index]
5:   end for
6:   position  $\leftarrow$  matrix × position
7:   dynamic_positions[i]  $\leftarrow$  matrix × positions[i]
8: end for

```

This algorithm uses **Linear Blend Skinning**, which is the standard skinning technique to its simplicity and efficiency [Jac+12]. We basically create a weighted sum of all the joint matrices used by the certain vertex and then multiply the resulting matrix with the vertex position to get the skinned position.

We have two implementations of the above skinning algorithm:

- **CPU skinning**

This implementation read all the vertex positions from the sub-buffer, runs the algorithm for each of them and stores the results in the respective sub-buffer for dynamic positions.

- **GPU skinning**

This implementation uses a vertex shader to run the algorithm for every vertex position and stores the resulting dynamic positions in a storage buffer. The pipeline that contains the vertex shader also disables rasterization, because no fragments should be generated during drawing and the pipeline should stop after running the vertex shader.

We could have also used a compute shader, which may perform better, but then we would need to implement a compute pipeline. With the approach we chose, we can simply repurpose the rasterization pipeline, which was already implemented.

Updating the acceleration structure

Algorithm 4 Updating acceleration structure

```
1: function UPDATE_ACCELERATION_STRUCTURE(scene)
2:   node_iterator = get_node_iterator(scene)
3:   tlas_instances = get_instances(scene.tlas)
4:   for all node in node_iterator do
5:     if has_blas(node) then
6:       if is_dynamic(node.blas) then
7:         update(node.blas)
8:       end if
9:       update_tlas_instance(tlas, node)
10:    end if
11:  end for
12:  update(scene.tlas)
13: end function
```

There is no `Scene::update_acceleration_structures` function, that you can call to update your acceleration structures. Instead, there are two slightly different member functions you can choose:

- `Scene::rebuild_acceleration_structures`
- `Scene::refit_acceleration_structures`

They both run more or less the same algorithm, the only difference is, that one uses "rebuilding" to update its acceleration structures and the other one uses "refitting".

5. Results and Discussion

6. Conclusion

In conclusion, when it comes to ray tracing in dynamic environments, there are a lot of variables we need to consider. However in our tests refitting is the clear winner, when it comes to updating our acceleration structures. Even though refitting does not save as much time as we initially anticipated, it is still considerably faster than rebuilding. Also, in none of our test scenes rebuilding resulted in a significant performance gain in the ray tracing step. This is probably because the animations in our scenes never change the mesh in a super dramatic way. In order for rebuilding to have a more impactful impact on the time spent on traversing the acceleration structure, we would probably need a much more dynamic scene, than any of the scenes we found.

A. General Addenda

If there are several additions you want to add, but they do not fit into the thesis itself, they belong here.

A.1. Detailed Addition

Even sections are possible, but usually only used for several elements in, e.g. tables, images, etc.

B. Figures

B.1. Example 1

✓

B.2. Example 2

✗

List of Figures

- 4.1. An example of how to use command buffers in your code. 9
- 4.2. Acceleration structure 12
- 4.3. Ray tracing pipeline 14
- 4.4. Shader Binding Table 15
- 4.5. Joint weight layout 19
- 4.6. Usage of sub-buffers in a scene 21

List of Tables

3.1. System Specifications 4

Bibliography

- [App68] A. Appel. “Some techniques for shading machine renderings of solids”. In: *Proceedings of the April 30–May 2, 1968, Spring Joint Computer Conference. AFIPS ’68 (Spring)*. Atlantic City, New Jersey: Association for Computing Machinery, 1968, pp. 37–45. ISBN: 9781450378970. DOI: 10.1145/1468075.1468082. URL: <https://doi.org/10.1145/1468075.1468082>.
- [Chr+18] P. Christensen, J. Fong, J. Shade, W. Wooten, B. Schubert, A. Kensler, S. Friedman, C. Kilpatrick, C. Ramshaw, M. Bannister, B. Rayner, J. Brouillat, and M. Liani. “An Advanced Path Tracing Architecture for Movie Rendering”. In: *ACM Transactions on Graphics*. Vol. 37. 3. 2018. URL: <https://docslib.org/doc/639313/an-advanced-path-tracing-architecture-for-movie-rendering>.
- [Cor18] N. Corporation. *Turing Architecture White Paper*. Tech. rep. NVIDIA Corporation, 2018.
- [HVV16] C. Hery, R. Villemin, and F. Hecht. *Towards Bidirectional Path Tracing at Pixar*. Tech. rep. 2016. URL: <https://graphics.pixar.com/library/BiDir/paper.pdf>.
- [Jac+12] A. Jacobson, I. Baran, L. Kavan, J. Popović, and O. Sorkine. “Fast Automatic Skinning Transformations”. In: *ACM Transactions on Graphics (proceedings of ACM SIGGRAPH)* 31.4 (2012), 77:1–77:10.
- [Khr25] Khronos Group. *Vulkan® 1.4.327 - A Specification (with all registered Vulkan extensions)*. The Khronos Group Inc. 2025. Chap. 39. Acceleration Structures. URL: <https://registry.khronos.org/vulkan/specs/latest/html/vkspec.html#acceleration-structure> (visited on 09/19/2025).
- [Koc+20] D. Koch, T. Hector, J. Barczak, and E. Werness. *Ray Tracing in Vulkan*. 2020. URL: <https://www.khronos.org/blog/ray-tracing-in-vulkan> (visited on 09/25/2025).
- [Lun25] C. Lunarg. *vk-bootstrap: Vulkan Bootstrapping Library*. 2025. URL: <https://github.com/charles-lunarg/vk-bootstrap> (visited on 09/19/2025).
- [Whi80] T. Whitted. “An improved illumination model for shaded display”. In: *Commun. ACM* 23.6 (June 1980), pp. 343–349. ISSN: 0001-0782. DOI: 10.1145/358876.358882. URL: <https://doi.org/10.1145/358876.358882>.